

PARTE TEORIA (12 PUNTI)

Domanda 1

Punteggio max.:
2,00

Contrassegna
domanda

Sia data una coda a priorità di dati inizialmente vuota implementata mediante uno heap. Sia data la sequenza di interi e carattere *:

20 32 19 * 41 * 28 * 54 39 71 * * 7 * 1

dove ad ogni intero corrisponde un inserimento nella coda a priorità e al carattere * un'estrazione con cancellazione del **massimo**. Si eseguano **in sequenza** le operazioni di cui sopra.

↴

A ▾

B

I

✎ ▾

☰

☰

☰

☰

🔗

🔄

😊

🖼️

👤

🔍

Formato della risposta:

- contenuto del vettore che implementa lo heap dopo l'inserzione di 28
- contenuto del vettore che implementa lo heap dopo l'inserzione di 71
- contenuto del vettore che implementa lo heap dopo l'inserzione di 1

Domanda 2

Punteggio max.:
2,50

Contrassegna
domanda

Data la catena di matrici (A_1, A_2, A_3, A_4) di dimensioni (3x6), (6x5), (5x4) e (4x2) rispettivamente, si determini mediante un algoritmo di programmazione dinamica la parentesizzazione ottima del prodotto di matrici che minimizza il numero di moltiplicazioni.

↴

A ▾

B

I

✎ ▾

☰

☰

☰

☰

🔗

🔄

😊

🖼️

👤

🔍

Domanda:

cosa contengono:

m[2][3]

s[2][3]

m[1][4]

s[1][4]

Formato della risposta:

m[2][3] = valore intero

s[2][3] = valore intero

m[1][4] = valore intero

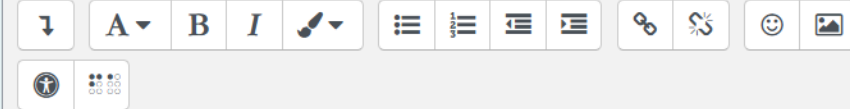
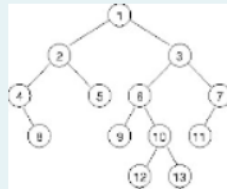
s[1][4] = valore intero

Domanda 3

Punteggio max.:
1,50

Contrassegna
domanda

Si visiti in pre-order, in-order e post-order il seguente albero binario:



Formato della risposta:

pre-order:

elenco chiavi

el1, el2, el3, ... eln

in-order:

elenco chiavi

el1, el2, el3, ... eln

post-order:

elenco chiavi

el1, el2, el3, ... eln

Domanda 4

Punteggio max.:
2,00

Contrassegna
domanda

Si inseriscano in sequenza le N=9 chiavi intere

138 48 103 60 213 28 98 70 215

in una tabella di hash (inizialmente vuota) gestita con open addressing con double hashing.

Si determini la dimensione minima M della tabella di per avere un fattore di carico $\alpha < 0.5$.

Formato della risposta:

dimensione minima della tabella

M =

Quale tra le seguenti funzioni di hash è opportuno usare?

$h_1(k) = k \% 17, h_2(k) = 1+k \% 17$

$h_1(k) = k \% 17, h_2(k) = 1+k \% 97$

$h_1(k) = k \% 19, h_2(k) = 1+k \% 19$

$h_1(k) = k \% 19, h_2(k) = 1+k \% 97$

Si motivi brevemente la risposta.

Collisioni chiave 98:

cella1 dove vi è collisione:

...

cellan dove vi è collisione:

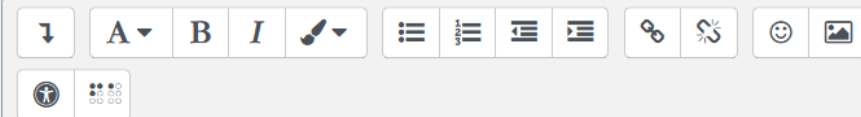
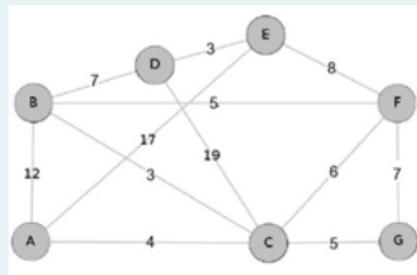
Al termine dell'inserzione, in quale cella si trova la chiave 215?

Domanda 5

Punteggio max.:
2,00

Contrassegna
domanda

Si determini mediante l'algoritmo di Kruskal un albero ricoprente minimo per il grafo non orientato, pesato e connesso in figura.



Formato della risposta:

Indicare se gli archi seguenti appartengono all'MST o meno:

- (B,C): sì/no
- (D,E): sì/no
- (B,F): sì/no
- (C,G): sì/no
- (B,D): sì/no
- (F,G): sì/no

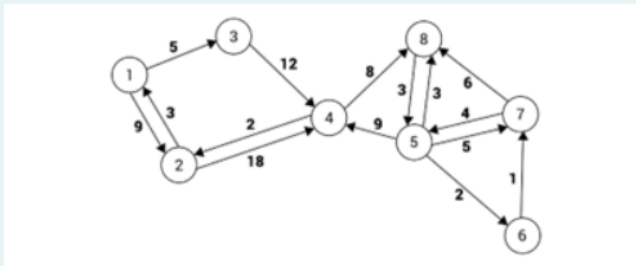
Peso minimo:

Domanda 6

Punteggio max.:
2,00

Contrassegna
domanda

Si determinino per il seguente grafo orientato pesato mediante l'algoritmo di Dijkstra i valori di tutti i cammini minimi che collegano il vertice **5** con ogni altro vertice. Qualora necessario, si trattino gli archi secondo l'ordine numerico.



↵

A ▾

B

I

✎ ▾

≡

≡

≡

≡

🔗

🔄

😊

🖼

🔍

🔍

Formato: contenuto della coda a priorità vertice/distanza minima ad ogni passo (inf per ∞)

Passo 0

PQ = {}

Passo 1

PQ = {}

....

Passo n

PQ = {}

Il risultato finale è ottimo? Sì/No

Breve motivazione:

Domanda 1

Punteggio max.: 2,00

Contrassegna domanda

Sia data una coda a priorità di dati inizialmente vuota implementata mediante uno heap. Sia data la sequenza di interi e carattere *:

20 32 19 * 41 * 28 * 54 39 71 * * 7 * 1

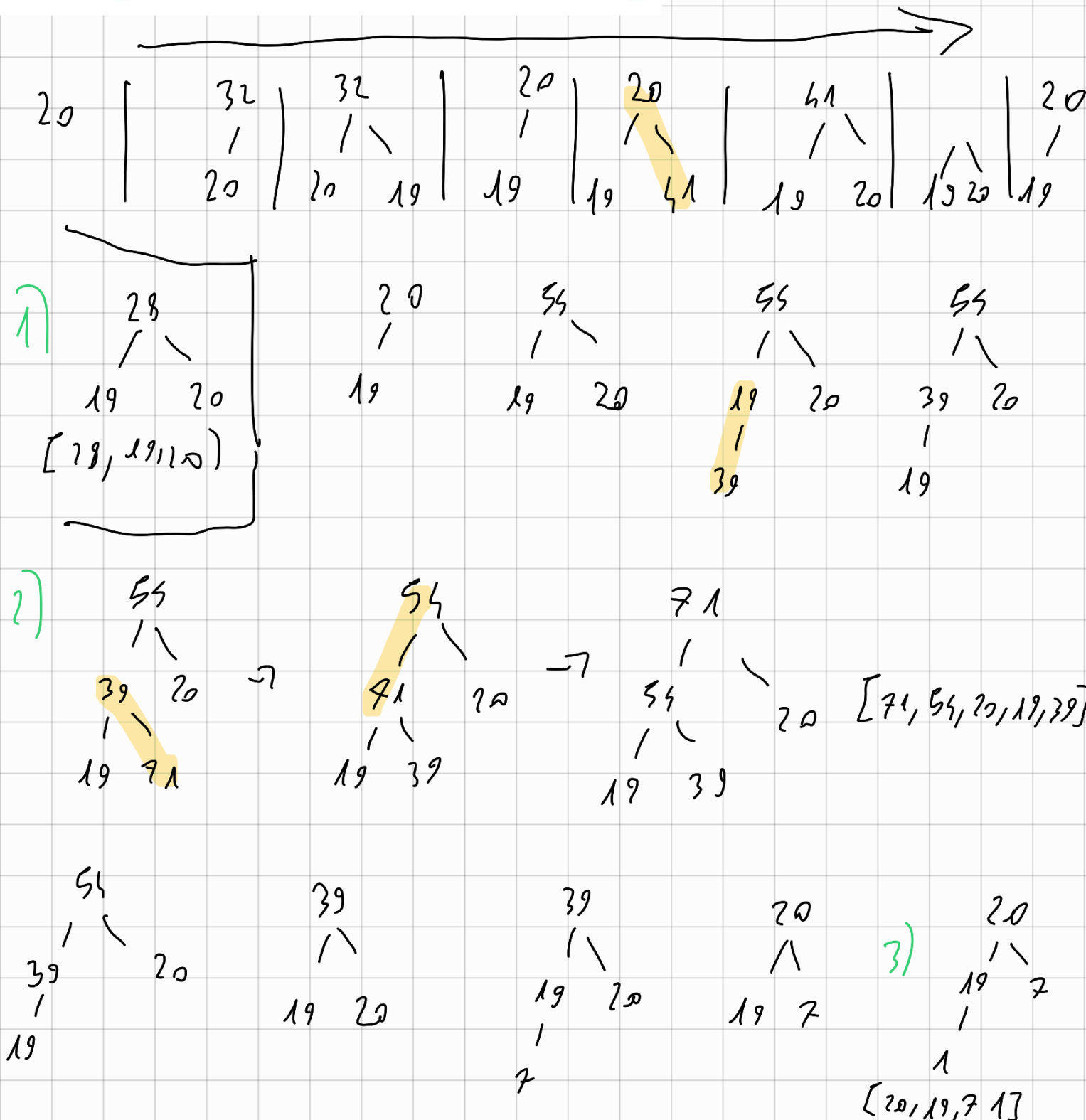
dove ad ogni intero corrisponde un inserimento nella coda a priorità e al carattere * un'estrazione con cancellazione del **massimo**. Si eseguano **in sequenza** le operazioni di cui sopra.

Cancellazione in coda a priorità

1. Si scambia l'elemento alla radice dell'albero con l'ultimo elemento presente nell'heap.
2. Si applica la trasformazione in heap, mantenendo l'elemento alla fine dell'heap appena scambiato nella stessa posizione.

Formato della risposta:

- contenuto del vettore che implementa lo heap dopo l'inserzione di 28
- contenuto del vettore che implementa lo heap dopo l'inserzione di 71
- contenuto del vettore che implementa lo heap dopo l'inserzione di 1

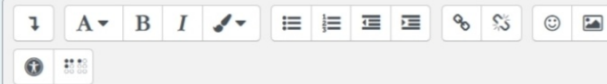


Domanda 2

Punteggio max:
2,50

Contrassegna domanda

Data la catena di matrici (A_1, A_2, A_3, A_4) di dimensioni (3×6) , (6×5) , (5×4) e (4×2) rispettivamente, si determini mediante un algoritmo di programmazione dinamica la parentesiizzazione ottima del prodotto di matrici che minimizza il numero di moltiplicazioni.



Domanda:

cosa contengono:

$m[2][3]$ 120

$s[2][3]$ 2

$m[1][4]$

$s[1][4]$

Formato della risposta:

$m[2][3]$ = valore intero

$s[2][3]$ = valore intero

$m[1][4]$ = valore intero

$s[1][4]$ = valore intero

Parentesiizzazione

1 di 10

- Definisci le matrici $A_1, A_2, \dots, A_n$, di dimensioni $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, con $p_0, p_1, \dots, p_n$ dimensioni delle matrici.
- Disegna la matrice superiore indicizzata da 1 a n per la valutazione della parentesiizzazione ottima, con le prime n celle con valore 0.
- Disegna la matrice superiore indicizzata da 2 a n per la stampa della soluzione, con le prime $n-1$ celle con valori da 1 a $n-1$.
- Definiamo i valori della parentesiizzazione per intervalli di dimensione 2, con $k = 1 + x + y$, con $x = \{0, 1, \dots, n-1\}$ e $y = \{0, 1, \dots, n-1\}$ e prendere i valori minori tra le scelte, inserendole all'indice $(2 + y, 1 + y)$ nella prima matrice.

$$m_{k(k+1)} = m_{kk} + m_{(k+1)(k+2)} + p_{k-1}p_kp_{k+1}$$

Nella seconda matrice, inserire l'indice k alla posizione $(2 + y, 1 + y)$.

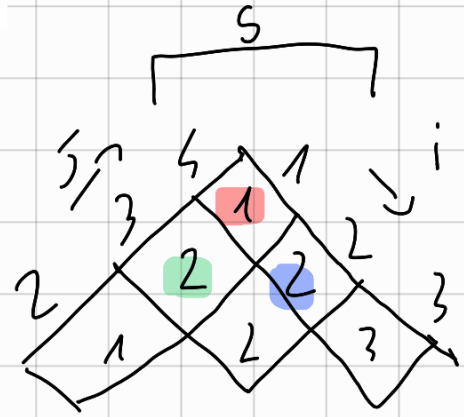
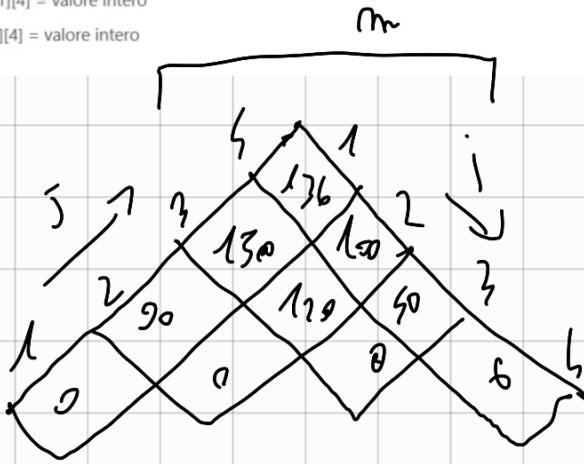
- Definiamo i valori della parentesiizzazione per intervalli di dimensione 3, con $k = 1 + x + y$, con $x = \{0, 1, \dots, n-2\}$ e $y = \{0, 1, \dots, n-2\}$ e prendere i valori minori tra le scelte, inserendole all'indice $(3 + y, 1 + y)$.

$$m_{k(k+2)} = \begin{cases} m_{kk} + m_{(k+1)(k+2)} + p_{k-1}p_kp_{k+2} \\ m_{k(k+1)} + m_{(k+2)(k+2)} + p_{k-1}p_{k+1}p_{k+2} \end{cases}$$

Nella seconda matrice, inserire l'indice k alla posizione $(3 + y, 1 + y)$.

- Ripetere fino a riempire le due matrici.
- Per stampare la soluzione, a partire dall'elemento $(n, 1)$, definendo il primo indice come i e il secondo indice come j , calcolare il valore $m = \lfloor (i + j)/2 \rfloor$.
 - Se $i \neq j$, ricorrere sugli indici (m, i) e $(j, m + 1)$.
 - Se $i = j$, stampare la matrice con indice i .
 - Ripetere fino a terminazione dell'albero di ricorsione.

p_0 3
 p_1 6
 p_2 5
 p_3 4
 p_4 2



Formula

$$m_{i,j} = m_{i,k} + m_{k+1,j} + p_{i-1} \cdot p_k \cdot p_j$$

L1

$$\cdot m_{11}, m_{22}, m_{33}, m_{44} = 0$$

L2

$$\cdot K:1 \mid m_{12} = m_{11} + m_{22} + p_0 \cdot p_1 \cdot p_2 = 90$$

$$\cdot K:2 \mid m_{23} = m_{22} + m_{33} + p_1 \cdot p_2 \cdot p_3 = 120$$

$$\cdot K:3 \mid m_{34} = m_{33} + m_{44} + p_2 \cdot p_3 \cdot p_4 = 60$$

L3

$$m_{13} \begin{cases} K:1 : 192 & \times \\ K:2 : 150 & \checkmark \end{cases}$$

$$m_{24} \begin{cases} K:2 : 100 & \checkmark \\ K:3 : 168 & \times \end{cases}$$

L4

$$m_{14} \begin{cases} K:1 : 136 & \checkmark \\ K:2 : 160 & \times \\ K:3 : 194 & \times \end{cases}$$

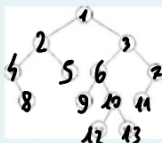
PARCHEGGIO STUPIDA $(A_1 A_2 A_3 A_4)$ (196 101 212)
 $((A_1) (A_2 A_3 A_4)) \rightarrow ((A_1) [(A_2) (A_3 A_4)])$

Domanda 3

Punteggio max:
1,50

Contrassegna
domanda

Si visiti in pre-order, in-order e post-order il seguente albero binario:



Formato della risposta:

pre-order: **1 2 4 5 3 6 9 10 12 13 7 11**

elenco chiavi

el1, el2, el3, ... eln

in-order: **4 8 2 5 1 9 6 12 10 13 3 11 7**

elenco chiavi

el1, el2, el3, ... eln

post-order: **8 4 5 2 9 12 13 10 6 11 7 3 1**

elenco chiavi

el1, el2, el3, ... eln

Visita pre-order, in-order, post-order

Tutte le visite partono dalla radice dell'albero.

1. La visita pre-order è definita attraverso la visita della radice, seguita dalla visita del figlio sinistro, seguita dalla visita del figlio destro.
2. La visita in-order è definita attraverso la visita del figlio sinistro, seguita dalla visita della radice, seguita dalla visita del figlio destro.
3. La visita post-order è definita attraverso la visita del figlio sinistro, seguita dalla visita del figlio destro, seguita dalla visita della radice.

Domanda 4

Punteggio max.: 2,00

Contrassegna domanda

Si inseriscano in sequenza le $N=9$ chiavi intere

138 48 103 60 213 28 98 70 215

in una tabella di hash (inizialmente vuota) gestita con open addressing con double hashing.

Si determini la dimensione minima M della tabella di per avere un fattore di carico $\alpha < 0.5$.

Formato della risposta:

dimensione minima della tabella
 $M =$

Quale tra le seguenti funzioni di hash è opportuno usare?

$h_1(k) = k \% 17, h_2(k) = 1 + k \% 17$ ☒
 $h_1(k) = k \% 17, h_2(k) = 1 + k \% 97$ ☒
 $h_1(k) = k \% 19, h_2(k) = 1 + k \% 19$ ☒
 $h_1(k) = k \% 19, h_2(k) = 1 + k \% 97$ ☒

Si motivi brevemente la risposta.

Collisioni chiave 98:
 cella1 dove vi è collisione:

 cella2 dove vi è collisione:

Al termine dell'inserzione, in quale cella si trova la chiave 215?

$$\alpha: \frac{N}{M} < \frac{1}{2}$$

$$M > 18 \approx 19$$

97 è coprimo di 19, L'UNICA
 COMBINAZIONE SCELTA

Double Hashing

1. Definiamo la funzione di Hashing, scegliendo $h_1(k)$ e $h_2(k)$ e A numero primo rispetto a M .

$$h_1(k) = K \% M$$

$$h_2(k) = 1 + K \% A$$

$$h(k) = [h_1(k) + i \cdot h_2(k)] \% M$$

2. Si applica la chiave di Hashing al valore k definito dalla funzione di Hashing, con inizialmente $i = 0$.

- Se la cella con indice $h(k)$ è vuota, allora inserisci il valore nella cella.
- Se non è vuota, allora tenta l'inserimento con $h_1(k) = h(k)$ e $i = i + 1$, e si ripete fino al successo.

$$\begin{aligned} & (h_1(k) + i \cdot h_2(k)) \% 19 \\ & \downarrow \qquad \qquad \downarrow \\ & (K \% 19) \quad (1 + K \% 97) \end{aligned}$$

$\cdot 138 \bmod 19 : 5$
 $\cdot 48 \qquad \qquad : 10$
 $\cdot 103 \qquad \qquad : 8$
 $\cdot 60 \qquad \qquad : 3$
 $\cdot 213 \qquad \qquad : 4$
 $\cdot 28 \qquad \qquad : 9$
 $\cdot 98 \bmod 19 : 3 \neq h_1(K)$

$h_2(K) = 1 + 98 \bmod 97 : 2$

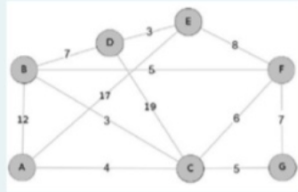
$\cdot i:1 \quad 3 + (1 \cdot 2) : 5 \quad \times$
 $\cdot i:2 \quad 3 + (2 \cdot 2) : 7 \quad \checkmark$

$\cdot 70 \qquad \qquad : 13$
 $\cdot 215 \qquad \qquad : 6$

} COLLISIONE

Domanda 5Punteggio max.:
2,00Contrassegna
domanda

Si determini mediante l'algoritmo di Kruskal un albero ricoprente minimo per il grafo non orientato, pesato e connesso in figura.

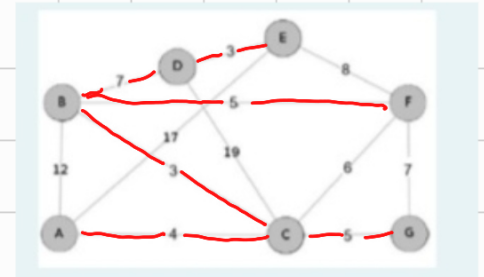


Formato della risposta:

Indicare se gli archi seguenti appartengono all'MST o meno:

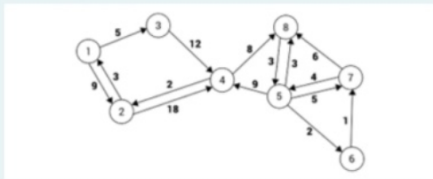
- (B,C): sì/no ✓
- (D,E): sì/no ✓
- (B,F): sì/no ✓
- (C,G): sì/no ✓
- (B,D): sì/no ✓
- (F,G): sì/no ✗

Peso minimo: 22

**Algoritmo di Kruskal**

1. Si inizia con l'insieme degli archi a pesi minori.
2. Si prendono tutti gli archi che non comprendono la creazione di un ciclo nel grafo.
3. Si ripete con il prossimo insieme degli archi a pesi minori fino ad avere un MST.

Si determinino per il seguente grafo orientato pesato mediante l'algoritmo di Dijkstra i valori di tutti i cammini minimi che collegano il vertice 5 con ogni altro vertice. Qualora necessario, si trattino gli archi secondo l'ordine numerico.



Formato: contenuto della coda a priorità vertice/distanza minima ad ogni passo (inf per ∞)

Passo 0
PQ = ={ }
Passo 1
PQ = ={ }
....
Passo n
PQ = ={ }

Il risultato finale è ottimo? Si/No SI

Breve motivazione:

Minori Pesi Minori

$P_0: 5/0, 1/\infty, 2/\infty, \dots$
 $P_1: 6/2, 8/3, 7/5, 4/9, \infty$ 5
 $P_2: 7/3, 8/3, 4/9, \infty$ 6
 $P_3: 8/3, 4/9, \infty$ 7
 $P_4: 4/9, \infty$ 8
 $P_5: 2/11, \infty$ 4
 $P_6: 1/14, \infty$ 2
 $P_7: 3/19$ 1
 $P_8:$ 3

Algoritmo di Dijkstra

1. Si inizia definendo una soluzione vuota, con una coda a priorità che contiene ogni nodo e il costo del cammino da un nodo x scelto verso ogni nodo, impostandolo inizialmente come ∞ e il costo del cammino dal nodo x al nodo x come 0.
2. Si estrae l'elemento a costo minore dalla coda a priorità e si rilassano gli archi, riordinando la coda a priorità per costi crescenti.
3. Si ripete fino ad avere la coda a priorità vuota, ed avendo quindi tutti i cammini minimi da x verso ogni altro nodo.

